

System Design Document

Society Maintenance Tracker

1. Project Overview

This project is a full-stack Society Maintenance Tracker designed to simplify complaint handling and society management. Residents can register complaints with photos, while administrators manage complaint status, publish notices, and monitor overdue issues. The application focuses on clean APIs, role-based authentication, and practical deployment using free cloud services.

2. Architecture

The system follows a three-tier architecture. The React frontend hosted on Vercel communicates with a Node.js and Express backend deployed on Render. The backend stores data in MongoDB Atlas and uploads complaint images to Cloudinary. Every client request passes through authentication middleware before reaching the business logic and database.

3. Core Modules

Authentication supports Resident and Admin roles using JWT. Complaint Management allows creation, status updates, priority handling, history tracking, overdue detection, and photo uploads. Flat Management maintains resident-flat mapping. Notice Board allows admins to publish important notices. Email integration sends notifications for complaint status changes and important notices using Nodemailer.

4. Database Design

MongoDB collections include Users, Societies, Flats, Complaints, and Notices. Complaint documents store title, description, priority, status, due date, Cloudinary image URL, resident, flat, society, and status history with timestamp, actor, and remarks.

5. API Flow

A request is received by Express routes, validated, authenticated using JWT, processed by controllers, stored through Mongoose models, and returned as JSON. Image uploads are handled before controller execution, while email notifications are triggered after successful updates.

6. Security

Passwords are hashed using bcrypt. JWT protects private endpoints. Environment variables store database credentials, Cloudinary keys, email configuration, and secret tokens. Sensitive information is excluded from GitHub using .gitignore.

7. Deployment

The frontend is deployed on Vercel, backend on Render, database on MongoDB Atlas, and media on Cloudinary. GitHub is used for version control and automatic deployment.

8. Conclusion

This project demonstrates a practical production-style backend with REST APIs, secure authentication, cloud storage, deployment, and modular code organization. The design is intentionally simple, maintainable, and suitable for academic evaluation while following common software engineering practices.